先进计算与数字工程研究中心

PATTERN RECOGNITION AND MACHINE LEARNING

Lecture 18 – Deep Learning IV: Sequence Models, Transformers, and New Advances



Chizhi Chris ZHANG

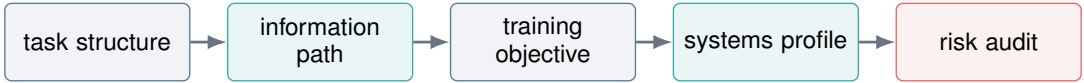
zhangchizhi@ciomp.ac.cn

Advanced Computing and Digital Technology Research Center

University of Chinese Academy of Sciences

Fall 2026

Choose an information path before choosing a model name



Architecture is a claim about which information must interact, persist, or be generated.

Guiding question

Should information move through recurrence, pairwise attention, latent variables, denoising, or selective state updates?

- recurrence compresses history
- attention retrieves by content
- generative objectives model uncertainty differently
- state spaces change long-sequence scaling

From recurrent states to evaluated foundation systems

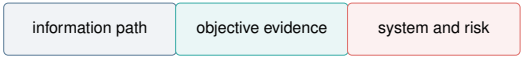
By the end

You can derive RNN and gated updates, implement attention and masks, explain Transformer blocks, compare VAE/GAN/diffusion objectives, and evaluate selective state-space and foundation-model systems.

Period	Core content	Decision
1	RNNs, BPTT, and gates	what must persist through time?
2	attention and Transformers	which positions must interact?
3	generative and new sequence models	which objective and system risk are acceptable?

Track information access, computation, and risk together

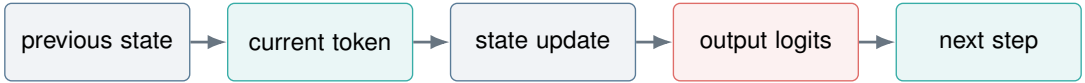
先进计算与数字工程研究中心
ADVANCED COMPUTING AND DIGITAL TECHNOLOGY RESEARCH CENTER



What to calculate

Every model records its causal mask, context length, training objective, token or sample budget, evaluation slices, calibration, memory, latency, safety tests, and rollback policy.

What information must a recurrent state carry from the past to predict the next output?



Parameter sharing handles variable length, while the hidden-state bottleneck must compress all task-relevant history.

Question

What information must a recurrent state carry from the past to predict the next output?

Example

predicting the next event in a sensor stream

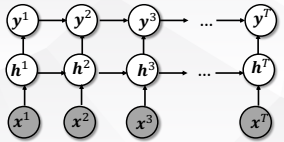
Main idea

Parameter sharing handles variable length, while the hidden-state bottleneck must compress all task-relevant history.

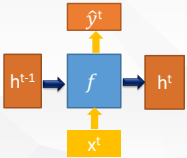
An RNN passes a hidden state through time

循环神经网络（Recurrent Neural Network，RNN)

- 隐含状态：存储与过去的相关信息
- 按时间展开



- 单个神经元



$$(\boldsymbol{y}^t, \boldsymbol{h}^t) = f(\boldsymbol{h}^{t-1}, \boldsymbol{x}^t)$$

模型与时间无关（时间上参数共享、时不变系统）

不管输入/输出序列有多长，我们只需要一个函数 f

Recurrent state and sequence likelihood: model and variables

Model and assumptions

An RNN updates a fixed-width hidden state from the previous state and current input, then maps that state to an output distribution.

$$h_t = \phi(W_h h_{t-1} + W_x x_t + b), \qquad p(y_t \mid x_{\le t}) = \mathrm{softmax}(U h_t)$$

Symbol	Meaning	Status
h_t	hidden state	summary of past input
W_h, W_x	shared transitions	same at every step
$p(y_t \mid x_{\le t})$	causal output	uses no future input

Recurrent state and sequence likelihood: worked example

Quantity	Value	Meaning
state width	128	fixed memory
sequence length	500	reused transition
outputs	500 distributions	one per step

What the result means

Use recurrence when compact state is plausible; test performance as dependency length grows.

Recurrent state and sequence likelihood: what to check

Main idea

Parameter sharing handles variable length, while the hidden-state bottleneck must compress all task-relevant history.

What to check

loss stratified by sequence length

Limit

a fixed hidden state can overwrite early information when the task requires precise long-range recall

Recurrent state and sequence likelihood: implementation

```
T, dx, dh = 6, 3, 4
Wx = rng.normal(scale=0.3, size=(dx, dh)); Wh = rng.normal(scale=0.3, size=(dh, dh)); bh = np.zeros(dh)
h = np.zeros(dh); states = []
for xt in sequence:
    h = np.tanh(xt @ Wx + h @ Wh + bh); states.append(h.copy())
states = np.stack(states)
print('RNN states:', states.shape, states[-1])
```

Evidence and failure check

Check loss stratified by sequence length with the fitted model and data split; reject the run if a fixed hidden state can overwrite early information when the task requires precise long-range recall.

Recurrent state and sequence likelihood: common mistakes

Example

predicting the next event in a sensor stream

Explain the calculation and check loss stratified by sequence length, using a small example.

Common mistake

a fixed hidden state can overwrite early information when the task requires precise long-range recall

What to do

Use recurrence when compact state is plausible; test performance as dependency length grows.

Recurrent state and sequence likelihood: check your understanding

Question

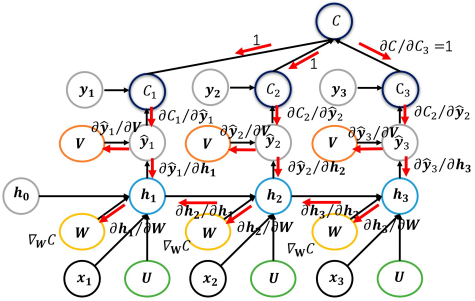
What information must a recurrent state carry from the past to predict the next output?

Answer

Parameter sharing handles variable length, while the hidden-state bottleneck must compress all task-relevant history.

Decision. Use recurrence when compact state is plausible; test performance as dependency length grows.

Why can a loss at a late time step create vanishing or exploding gradients far in the past?



Question

Why can a loss at a late time step create vanishing or exploding gradients far in the past?

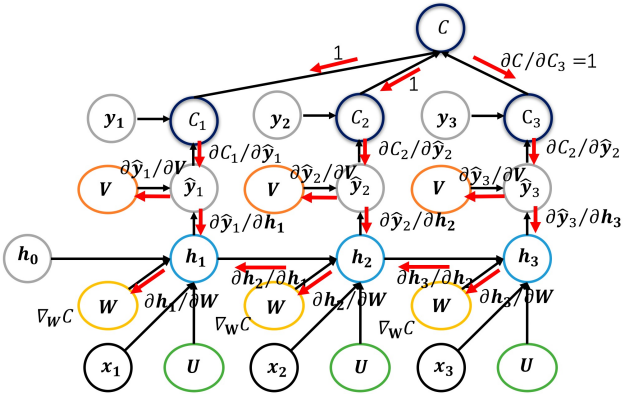
Example

learning a dependency between the first and hundredth token

Main idea

Long-range gradients multiply many Jacobians; their singular values determine contraction or expansion.

Backpropagation follows the unfolded sequence



Unfold the shared parameters through time, then add their gradient contributions.

Backpropagation through time: model and variables

Model and assumptions

BPTT unrolls the shared recurrent computation and applies reverse-mode differentiation through every state transition.

$$\frac{\partial \mathcal{L}_t}{\partial h_k} = \frac{\partial \mathcal{L}_t}{\partial h_t} \prod_{j=k+1}^{t} \frac{\partial h_j}{\partial h_{j-1}}$$

Symbol	Meaning	Status
$\partial h_j / \partial h_{j-1}$	transition Jacobian	local gradient transform
$t - k$	dependency length	number of products
truncation	limited unroll	compute-memory tradeoff

Backpropagation through time: worked example

Quantity	Value	Meaning
Jacobian scale	0.8	contractive
twenty steps	$0.8^{20} = 0.012$	vanishing
scale 1.2	$1.2^{20} = 38.3$	exploding

What the result means

Use clipping for stability and gates, attention, or alternative state models for persistent long-range credit assignment.

The choice rule for Backpropagation through time

Main idea

Long-range gradients multiply many Jacobians; their singular values determine contraction or expansion.

What to check

gradient norm versus temporal distance

Limit

gradient clipping prevents numerical explosion but does not restore a vanished learning signal

Backpropagation through time: implementation

1. unroll a short sequence
2. cache states
3. accumulate parameter gradients over time
4. monitor gradient norm
5. clip and test longer dependencies

Check the result

Check gradient norm versus temporal distance on training and held-out examples.

Common mistake

gradient clipping prevents numerical explosion but does not restore a vanished learning signal

Backpropagation through time: common mistakes

先进计算与数字工程研究中心
ADVANCED COMPUTING AND DIGITAL TECHNOLOGY RESEARCH CENTER

Example

learning a dependency between the first and hundredth token

Explain the calculation and check gradient norm versus temporal distance, using a small example.

Common mistake

gradient clipping prevents numerical explosion but does not restore a vanished learning signal

What to do

Use clipping for stability and gates, attention, or alternative state models for persistent long-range credit assignment.

Backpropagation through time: check your understanding

Question

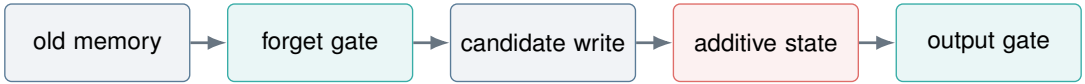
Why can a loss at a late time step create vanishing or exploding gradients far in the past?

Answer

Long-range gradients multiply many Jacobians; their singular values determine contraction or expansion.

Decision. Use clipping for stability and gates, attention, or alternative state models for persistent long-range credit assignment.

How do gates create a controllable path for retaining, writing, and exposing memory?



The LSTM cell offers an additive memory path; GRU merges gates for a simpler state update.

Question

How do gates create a controllable path for retaining, writing, and exposing memory?

Example

retaining a regime indicator across many sensor readings

Main idea

The LSTM cell offers an additive memory path; GRU merges gates for a simpler state update.

LSTM gates control retention, writing, and reading

- $f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$: how much old memory remains.
- $i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$: how much new information is written.
- $c_t = f_t \odot c_{t-1} + i_t \odot \widetilde{c}_t$.
- $h_t = o_t \odot \tanh(c_t)$: an output gate controls what is read out.

LSTM and GRU gates: model and variables

Model and assumptions

Gated RNNs use sigmoid-controlled interpolation to preserve or update state components over time.

$$c_t = f_t \odot c_{t-1} + i_t \odot \widetilde{c}_t, \qquad h_t = o_t \odot \tanh(c_t)$$

Symbol	Meaning	Status
f_t	forget gate	retain old memory
i_t	input gate	write candidate
o_t	output gate	expose cell state

LSTM and GRU gates: worked example

Quantity	Value	Meaning
old cell	2.0	stored value
forget / input	$(0.9, 0.2)$	retain and write
candidate	0.5	new cell 1.9

What the result means

Use gated recurrence when sequential state updates are natural, and inspect gates rather than assuming memory works.

LSTM and GRU gates: checking the result

Main idea

The LSTM cell offers an additive memory path; GRU merges gates for a simpler state update.

What to check

long-delay recall and gate saturation

Limit

gates saturated near zero or one can freeze learning even when the forward memory appears stable

What to record when running LSTM and GRU gates

1. write every gate shape
2. initialize forget bias carefully
3. inspect gate saturation
4. compare LSTM and GRU at matched width
5. stratify by dependency length

Check the result

Check long-delay recall and gate saturation on training and held-out examples.

Common mistake

gates saturated near zero or one can freeze learning even when the forward memory appears stable

The warning sign for LSTM and GRU gates

Example

retaining a regime indicator across many sensor readings

Explain the calculation and check long-delay recall and gate saturation, using a small example.

Common mistake

gates saturated near zero or one can freeze learning even when the forward memory appears stable

What to do

Use gated recurrence when sequential state updates are natural, and inspect gates rather than assuming memory works.

Explain LSTM and GRU gates without naming the algorithm

Question

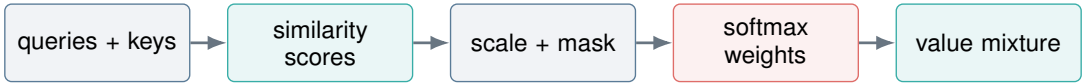
How do gates create a controllable path for retaining, writing, and exposing memory?

Answer

The LSTM cell offers an additive memory path; GRU merges gates for a simpler state update.

Decision. Use gated recurrence when sequential state updates are natural, and inspect gates rather than assuming memory works.

How can each token retrieve content from all candidate positions?



The query-key geometry chooses where to read; values carry the information that is actually aggregated.

Question

How can each token retrieve content from all candidate positions?

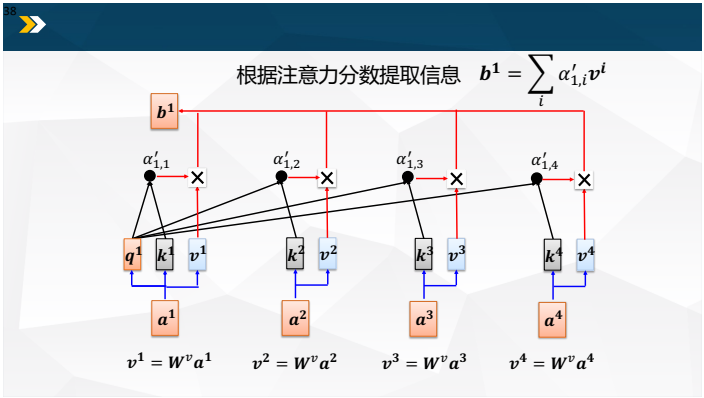
Example

linking a pronoun to a distant entity mention

Main idea

The query-key geometry chooses where to read; values carry the information that is actually aggregated.

Attention returns a weighted sum of values



How Scaled dot-product attention changes the representation

Model and assumptions

Attention compares a query with keys, normalizes compatibility scores, and forms a weighted sum of values.

$$\mathrm{Attn}(Q, K, V) = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right) V$$

Symbol	Meaning	Status
Q	queries	what each position seeks
K	keys	how positions are addressed
V	values	retrieved content

Check the direction of change in Scaled dot-product attention

Quantity	Value	Meaning
scores	$(2, 1, 0)$	one query
softmax	$(.665, .245, .090)$	attention weights
output	weighted values	content retrieval

What the result means

Use attention for flexible content interaction, but validate explanations with interventions.

When Scaled dot-product attention earns a place in the pipeline

Main idea

The query-key geometry chooses where to read; values carry the information that is actually aggregated.

What to check

task loss and attention invariance tests

Limit

attention weights are model routing coefficients, not automatically faithful explanations of causal importance

Scaled dot-product attention: implementation

先进计算与数字工程研究中心
ADVANCED COMPUTING AND DIGITAL TECHNOLOGY RESEARCH CENTER

```
def attention(Q, K, V, mask=None):
    scores = Q @ K.T / np.sqrt(Q.shape[-1])
    if mask is not None: scores = np.where(mask, scores, -1e9)
    scores -= scores.max(axis=-1, keepdims=True)
    weights = np.exp(scores); weights /= weights.sum(axis=-1, keepdims=True)
    return weights @ V, weights
attn_out, attn_w = attention(Q, K, V)
print('Attention rows/output:', attn_w.sum(axis=1), attn_out.shape)
```

Evidence and failure check

Check task loss and attention invariance tests with the fitted model and data split; reject the run if attention weights are model routing coefficients, not automatically faithful explanations of causal importance.

When the objective of Scaled dot-product attention is not enough

Example

linking a pronoun to a distant entity mention

Explain the calculation and check task loss and attention invariance tests, using a small example.

Common mistake

attention weights are model routing coefficients, not automatically faithful explanations of causal importance

What to do

Use attention for flexible content interaction, but validate explanations with interventions.

A final decision on Scaled dot-product attention

Question

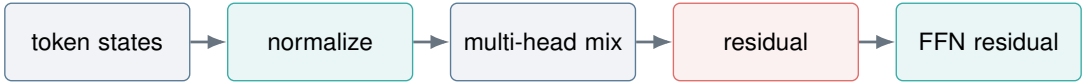
How can each token retrieve content from all candidate positions?

Answer

The query-key geometry chooses where to read; values carry the information that is actually aggregated.

Decision. Use attention for flexible content interaction, but validate explanations with interventions.

How does a Transformer alternate token interaction with per-token computation?



Attention mixes information across positions; the feed-forward network transforms channels independently at each position.

Question

How does a Transformer alternate token interaction with per-token computation?

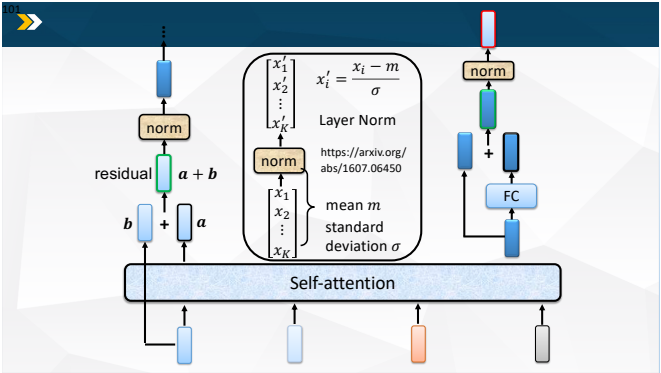
Example

encoding a document with global cross-sentence interactions

Main idea

Attention mixes information across positions; the feed-forward network transforms channels independently at each position.

A Transformer block combines attention and a feedforward layer



The variables that make Transformer blocks and multi-head interaction work

Model and assumptions

A Transformer block combines multi-head self-attention, positionwise feed-forward layers, residual paths, and normalization.

$$H' = H + \mathrm{MHA}(\mathrm{LN}(H)), \qquad H'' = H' + \mathrm{FFN}(\mathrm{LN}(H'))$$

Symbol	Meaning	Status
MHA	multi-head attention	token interaction
FFN	positionwise nonlinear map	channel transformation
LN	layer normalization	per-token stabilization

Read the numbers produced by Transformer blocks and multi-head interaction

Quantity	Value	Meaning
model width	512	representation
eight heads	64 per head	parallel subspaces
FFN width	2048	channel expansion

What the result means

Use full attention when global interaction is worth quadratic sequence cost; otherwise introduce structure or sparsity.

Three conditions for trusting Transformer blocks and multi-head interaction

Main idea

Attention mixes information across positions; the feed-forward network transforms channels independently at each position.

What to check

validation loss versus tokens and memory

Limit

quadratic attention memory can dominate before parameter count becomes the system bottleneck

An implementation checklist for Transformer blocks and multi-head interaction

1. fix model and head dimensions
2. select pre-norm or post-norm
3. implement residual paths
4. mask by task
5. profile sequence-length cost

Check the result

Check validation loss versus tokens and memory on training and held-out examples.

Common mistake

quadratic attention memory can dominate before parameter count becomes the system bottleneck

Reject Transformer blocks and multi-head interaction under this condition

Example

encoding a document with global cross-sentence interactions

Explain the calculation and check validation loss versus tokens and memory, using a small example.

Common mistake

quadratic attention memory can dominate before parameter count becomes the system bottleneck

What to do

Use full attention when global interaction is worth quadratic sequence cost; otherwise introduce structure or sparsity.

Transformer blocks and multi-head interaction: check your understanding

Question

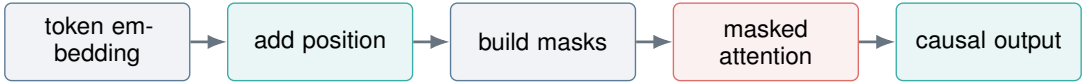
How does a Transformer alternate token interaction with per-token computation?

Answer

Attention mixes information across positions; the feed-forward network transforms channels independently at each position.

Decision. Use full attention when global interaction is worth quadratic sequence cost; otherwise introduce structure or sparsity.

How does a permutation-equivariant attention layer learn order without seeing the future?



These masks encode different semantics and must be composed before score normalization.

Question

How does a permutation-equivariant attention layer learn order without seeing the future?

Example

next-token prediction on padded variable-length batches

Main idea

These masks encode different semantics and must be composed before score normalization.

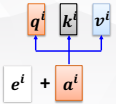
Position information preserves word order

先进计算与数字工程研究中心
ADVANCED COMPUTING AND DIGITAL TECHNOLOGY RESEARCH CENTER

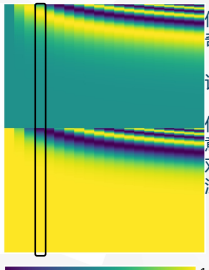
13

位置编码（Positional Encoding）

- 自注意力中没有位置信息。
- 每个位置对应一个唯一的位置向量e^i
- 从数据中学习
- 旋转位置编码（**RoPE**）：将绝对位置信息以旋转矩阵的形式融入Query和Key中。LLaMA、GPT-NeoX等众多现代大模型都采用了RoPE。



每列表示一个位置编码 e^i



偶数位置：sin函数
奇数位置：cos函数

计算注意力分数（点积）时，相对位置的变化会对注意力产生影响，绝对位置变化不会对注意力产生影响。

Position, causality, and padding masks: model and variables

Model and assumptions

Position representations inject order, causal masks block future keys, and padding masks remove nonexistent tokens.

$$p_{t,2i} = \sin(t/10000^{2i/d}), \quad p_{t,2i+1} = \cos(t/10000^{2i/d}), \quad M_{t,s} = -\infty \text{ if } s > t$$

Symbol	Meaning	Status
p_t	position representation	order signal
$M_{t,s}$	attention mask	allowed information path
padding mask	valid-token indicator	ignores padded entries

先进计算与数字工程研究中心
ADVANCED COMPUTING AND DIGITAL TECHNOLOGY RESEARCH CENTER

Position, causality, and padding masks: worked example

Quantity	Value	Meaning
query position	$t = 3$	fourth token
allowed keys	$s \leq 3$	past and current
future weights	0	after softmax

What the result means

Require a unit test proving earlier predictions are invariant to future-token changes.

A decision card for Position, causality, and padding masks

Main idea

These masks encode different semantics and must be composed before score normalization.

What to check

change in earlier logits after perturbing future tokens

Limit

an inverted or late-applied mask leaks future targets while training loss looks exceptionally good

Reproduce the result of Position, causality, and padding masks

```
causal = np.tril(np.ones((len(Q), len(K)), dtype=bool))
causal_out, causal_w = attention(Q, K, V, causal)
print('Causal upper-triangle mass:', float(np.triu(causal_w, 1).sum()))
```


Evidence and failure check

Check change in earlier logits after perturbing future tokens with the fitted model and data split; reject the run if an inverted or late-applied mask leaks future targets while training loss looks exceptionally good.

A stress test for Position, causality, and padding masks

Example

next-token prediction on padded variable-length batches

Explain the calculation and check change in earlier logits after perturbing future tokens, using a small example.

Common mistake

an inverted or late-applied mask leaks future targets while training loss looks exceptionally good

What to do

Require a unit test proving earlier predictions are invariant to future-token changes.

Position, causality, and padding masks: check your understanding

Question

How does a permutation-equivariant attention layer learn order without seeing the future?

Answer

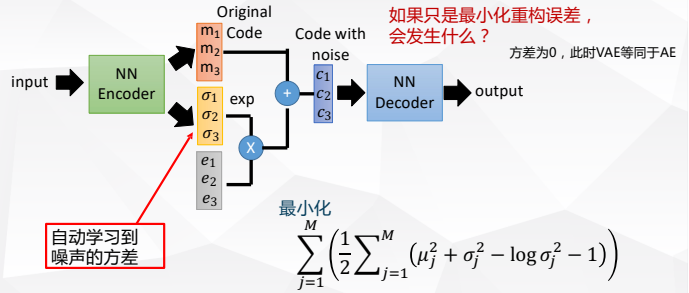
These masks encode different semantics and must be composed before score normalization.

Decision. Require a unit test proving earlier predictions are invariant to future-token changes.

A VAE learns an encoder and a probabilistic decoder

先进计算与数字工程研究中心
ADVANCED COMPUTING AND DIGITAL TECHNOLOGY RESEARCH CENTER

Why VAE



VAE training balances reconstruction and a latent prior

先进计算与数字工程研究中心
ADVANCED COMPUTING AND DIGITAL TECHNOLOGY RESEARCH CENTER

l_b的第一项$\mathrm{KL}\big(q(\boldsymbol{z}|\boldsymbol{x})||p(\boldsymbol{z})\big)$

两个高斯分布的KL散度

最小化 $\mathrm{KL}\big(q(\boldsymbol{z}|\boldsymbol{x})||p(\boldsymbol{z})\big)$



最好没有p(z)=0而q(z|x)！=0的情况，否则会无穷大

两个高斯分布的KI散度（ M 维 ）：

$$\mathrm{KL}\big(N(\boldsymbol{\mu}_1,\boldsymbol{\Sigma}_1)||N(\boldsymbol{\mu}_2,\boldsymbol{\Sigma}_2)\big)=\frac{1}{2}\left[\ln\frac{\det\boldsymbol{\Sigma}_2}{\det\boldsymbol{\Sigma}_1}-M+\mathrm{tr}(\boldsymbol{\Sigma}_2^{-1}\boldsymbol{\Sigma}_1)+(\boldsymbol{\mu}_2-\boldsymbol{\mu}_1)^{\mathrm{T}}\boldsymbol{\Sigma}_2^{-1}(\boldsymbol{\mu}_2-\boldsymbol{\mu}_1)\right]$$

所以，$p(\boldsymbol{z})\sim N(0,\boldsymbol{I})$，$\boldsymbol{\Sigma}=\mathrm{diag}[\sigma_1^2,\sigma_2^2,\ldots,\sigma_M^2]$

$$\mathrm{KL}\big(N(\boldsymbol{\mu},\boldsymbol{\Sigma})||N(0,\boldsymbol{I})\big)=\frac{1}{2}\sum_{j=1}^{M}\big(\mu_j^2+\sigma_j^2-\log\sigma_j^2-1\big)$$

Variational autoencoders: model and variables

先进计算与数字工程研究中心
ADVANCED COMPUTING AND DIGITAL TECHNOLOGY RESEARCH CENTER

Model and assumptions

A VAE jointly trains an encoder distribution and decoder by maximizing an evidence lower bound with a reconstruction term and prior-matching KL term.

$$\log p_\theta(x) \geq \mathbb{E}_{q_\phi(z|x)} \log p_\theta(x \mid z) - D_{KL}(q_\phi(z \mid x) \| p(z))$$

Symbol	Meaning	Status
$q_\phi(z \mid x)$	encoder posterior	approximate inference
$p_\theta(x \mid z)$	decoder likelihood	reconstruction model
$p(z)$	latent prior	sampling reference

ADVANCED COMPUTING AND DIGITAL TECHNOLOGY RESEARCH CENTER

Variational autoencoders: worked example

Quantity	Value	Meaning
μ	0.5	latent mean
σ	0.2	latent scale
$\epsilon = -1$	$z = 0.3$	reparameterized sample

What the result means

Diagnose posterior collapse and tune capacity or KL scheduling before interpreting the latent space.

Variational autoencoders: checking the result

Main idea

The latent variable trades reconstruction fidelity against a smooth, sampleable representation.

What to check

held-out ELBO, reconstruction quality, and active latent dimensions

Limit

a powerful decoder can ignore the latent code, driving the posterior toward the prior

Variational autoencoders: implementation

```
mu = np.array([0.5, -0.2]); logvar = np.array([-1.0, 0.4]); eps = rng.normal(size=2)
z = mu + np.exp(0.5*logvar)*eps
kl = -0.5*np.sum(1 + logvar - mu*mu - np.exp(logvar))
print('VAE latent/KL:', z, round(float(kl), 4))
```


Evidence and failure check

Check held-out ELBO, reconstruction quality, and active latent dimensions with the fitted model and data split; reject the run if a powerful decoder can ignore the latent code, driving the posterior toward the prior.

What breaks first in Variational autoencoders

Example

learning a low-dimensional representation of handwritten symbols

Explain the calculation and check held-out ELBO, reconstruction quality, and active latent dimensions, using a small example.

Common mistake

a powerful decoder can ignore the latent code, driving the posterior toward the prior

What to do

Diagnose posterior collapse and tune capacity or KL scheduling before interpreting the latent space.

Variational autoencoders: check your understanding

Question

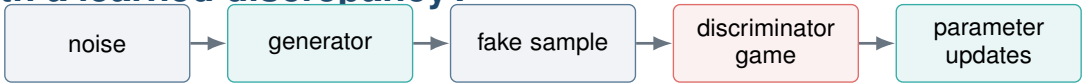
How can a model learn a continuous latent representation while retaining a tractable training objective?

Answer

The latent variable trades reconstruction fidelity against a smooth, sampleable representation.

Decision. Diagnose posterior collapse and tune capacity or KL scheduling before interpreting the latent space.

How does an adversarial game replace an explicit likelihood with a learned discrepancy?



The discriminator supplies a learned training signal, but the coupled game can cycle, collapse modes, or hide poor coverage behind sharp samples.

Question

How does an adversarial game replace an explicit likelihood with a learned discrepancy?

Example

generating diverse visual prototypes from limited data

Main idea

The discriminator supplies a learned training signal, but the coupled game can cycle, collapse modes, or hide poor coverage behind sharp samples.

A GAN trains a generator against a discriminator



Generative adversarial networks: model and variables

先进计算与数字工程研究中心
ADVANCED COMPUTING AND DIGITAL TECHNOLOGY RESEARCH CENTER

Model and assumptions

A GAN trains a discriminator to separate real from generated samples and a generator to produce samples that fool it.

$$\min_G \max_D \; \mathbb{E}_{x\sim p_{data}} \log D(x) + \mathbb{E}_{z\sim p(z)} \log[1 - D(G(z))]$$

Symbol	Meaning	Status
G	generator	maps noise to samples
D	discriminator	learned realness score
$p(z)$	noise prior	source of variation

Generative adversarial networks: worked example

Quantity	Value	Meaning
real score	$D(x) = 0.9$	discriminator target one
fake score	$D(G(z)) = 0.2$	easy fake
generator goal	raise fake score	adversarial signal

What the result means

Accept a GAN only when coverage, fidelity, and memorization checks all pass across seeds.

Accept or reject Generative adversarial networks

Main idea

The discriminator supplies a learned training signal, but the coupled game can cycle, collapse modes, or hide poor coverage behind sharp samples.

What to check

distributional distance, precision-recall for generation, and memorization audit

Limit

mode collapse produces visually sharp but repetitive samples that omit large parts of the data distribution

Generative adversarial networks: implementation

1. normalize data consistently
2. balance discriminator and generator updates
3. track fixed-noise samples
4. measure diversity and fidelity
5. inspect nearest training examples

Check the result

Check distributional distance, precision-recall for generation, and memorization audit on training and held-out examples.

Common mistake

mode collapse produces visually sharp but repetitive samples that omit large parts of the data distribution

The hidden assumption in Generative adversarial networks

Example

generating diverse visual prototypes from limited data

Explain the calculation and check distributional distance, precision-recall for generation, and memorization audit, using a small example.

Common mistake

mode collapse produces visually sharp but repetitive samples that omit large parts of the data distribution

What to do

Accept a GAN only when coverage, fidelity, and memorization checks all pass across seeds.

Generative adversarial networks: check your understanding

先进计算与数字工程研究中心
ADVANCED COMPUTING AND DIGITAL TECHNOLOGY RESEARCH CENTER

Question

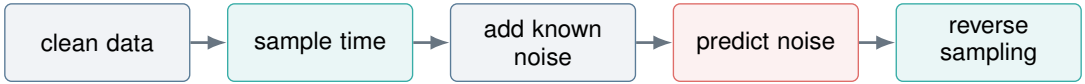
How does an adversarial game replace an explicit likelihood with a learned discrepancy?

Answer

The discriminator supplies a learned training signal, but the coupled game can cycle, collapse modes, or hide poor coverage behind sharp samples.

Decision. Accept a GAN only when coverage, fidelity, and memorization checks all pass across seeds.

How can a model learn generation by reversing a gradual noising process?



Training uses random noise levels; sampling integrates many learned reverse steps and exposes a quality-speed tradeoff.

Question

How can a model learn generation by reversing a gradual noising process?

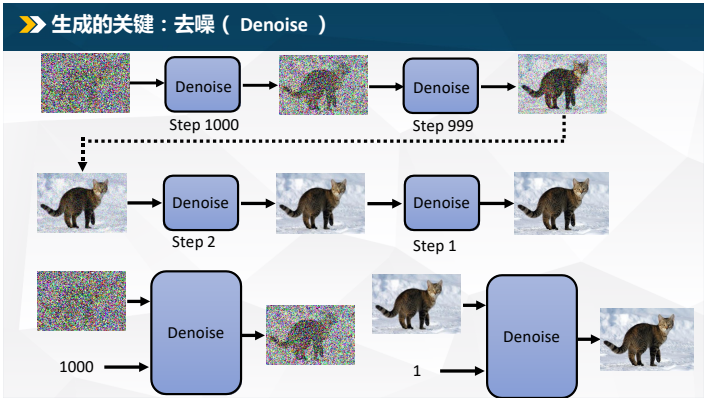
Example

conditional generation under a fixed inference budget

Main idea

Training uses random noise levels; sampling integrates many learned reverse steps and exposes a quality-speed tradeoff.

Diffusion generates by repeated denoising



Diffusion and score-based generation: model and variables

先进计算与数字工程研究中心
ADVANCED COMPUTING AND DIGITAL TECHNOLOGY RESEARCH CENTER

Model and assumptions

Diffusion models corrupt data through a known Gaussian process and train a network to predict noise, score, or clean data for iterative denoising.

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon, \qquad \mathcal{L} = \mathbb{E}\|\epsilon - \epsilon_\theta(x_t, t, c)\|_2^2$$

Symbol	Meaning	Status
$\bar{\alpha}_t$	signal schedule	remaining data fraction
ϵ	Gaussian noise	known training target
c	conditioning	optional guidance input

Diffusion and score-based generation: worked example

Quantity	Value	Meaning
$\bar{\alpha}_t$	0.64	signal power
$x_0 = 1, \epsilon = -0.5$	one dimension	training pair
x_t	0.5	corrupted value

What the result means

Choose sampler and guidance from a Pareto curve of quality, diversity, safety, and latency.

Separate a good fit from a good decision in Diffusion and score-based generation

Main idea

Training uses random noise levels; sampling integrates many learned reverse steps and exposes a quality-speed tradeoff.

What to check

fidelity, diversity, conditional alignment, and sampling latency

Limit

strong guidance can improve apparent prompt alignment while reducing diversity or amplifying artifacts

Diffusion and score-based generation: implementation

先进计算与数字工程研究中心
ADVANCED COMPUTING AND DIGITAL TECHNOLOGY RESEARCH CENTER

```
x0 = np.array([1.0, -0.5, 0.2]); alpha_bar = 0.64; noise = rng.normal(size=x0.shape)
xt = np.sqrt(alpha_bar)*x0 + np.sqrt(1-alpha_bar)*noise
x0_recovered = (xt - np.sqrt(1-alpha_bar)*noise)/np.sqrt(alpha_bar)
print('Diffusion reconstruction error:', float(np.max(np.abs(x0-x0_recovered))))
```


Evidence and failure check

Check fidelity, diversity, conditional alignment, and sampling latency with the fitted model and data split; reject the run if strong guidance can improve apparent prompt alignment while reducing diversity or amplifying artifacts.

Do not trust Diffusion and score-based generation before this check

Example

conditional generation under a fixed inference budget

Explain the calculation and check fidelity, diversity, conditional alignment, and sampling latency, using a small example.

Common mistake

strong guidance can improve apparent prompt alignment while reducing diversity or amplifying artifacts

What to do

Choose sampler and guidance from a Pareto curve of quality, diversity, safety, and latency.

Diffusion and score-based generation: check your understanding

Question

How can a model learn generation by reversing a gradual noising process?

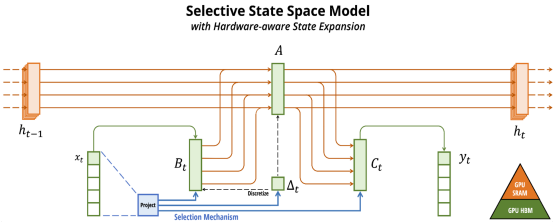
Answer

Training uses random noise levels; sampling integrates many learned reverse steps and exposes a quality-speed tradeoff.

Decision. Choose sampler and guidance from a Pareto curve of quality, diversity, safety, and latency.

先进计算与数字工程研究中心
ADVANCED COMPUTING AND DIGITAL TECHNOLOGY RESEARCH CENTER

Selective state models process long sequences with compact memory



Question

When can a linear-time selective state model replace quadratic attention, and what evidence is still required at scale?

Example

serving long-document assistance under memory and safety constraints

Main idea

Selection lets content control what the state retains; linear sequence scaling changes the systems tradeoff but does not solve factuality, misuse, privacy, or distribution shift.

Mamba selects what to retain from each input

广州计算与数字工程研究中心
ADVANCED COMPUTING AND DIGITAL TECHNOLOGY RESEARCH CENTER

Mamba : a Selective SSM

Algorithm 1 SSM (S4)

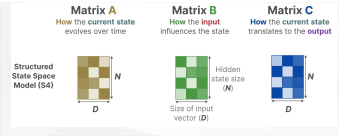
Input: x : (B, L, D)
Output: y : (B, L, D)
1: $\boldsymbol{A}$: (D, N) ← Parameter
▷ Represents structured $N \times N$ matrix
2: $\boldsymbol{B}$: (D, N) ← Parameter
3: $\boldsymbol{C}$: (D, N) ← Parameter
4: Δ : (D) ← τ_Δ(Parameter)
5: $\overline{\boldsymbol{A}}, \overline{\boldsymbol{B}}$: (D, N) ← discretize($\Delta, \boldsymbol{A}, \boldsymbol{B}$)
6: $y \leftarrow \mathrm{SSM}(\overline{\boldsymbol{A}}, \overline{\boldsymbol{B}}, \boldsymbol{C})(x)$
▷ Time-invariant: recurrence or convolution
7: **return** y

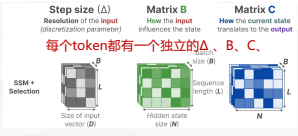
Algorithm 2 SSM + Selection (S6)

Input: x : (B, L, D)
Output: y : (B, L, D)
1: $\boldsymbol{A}$: (D, N) ← Parameter
▷ Represents structured $N \times N$ matrix
2: $\boldsymbol{B}$: (B, L, N) ← $s_B(x)$
3: $\boldsymbol{C}$: (B, L, N) ← $s_C(x)$
4: Δ : (B, L, D) ← τ_Δ(Parameter+$s_\Delta(x)$)
5: $\overline{\boldsymbol{A}}, \overline{\boldsymbol{B}}$: (B, L, D, N) ← discretize($\Delta, \boldsymbol{A}, \boldsymbol{B}$)
6: $y \leftarrow \mathrm{SSM}(\overline{\boldsymbol{A}}, \overline{\boldsymbol{B}}, \boldsymbol{C})(x)$
▷ Time-varying: recurrence (*scan*) only
7: **return** y

输入数据依赖

A不和输入有关，但$\overline{\text{A}}$通过 Δ与输入有关





Selective state spaces, Mamba, and foundation-model evaluation: model and variables

Model and assumptions

Selective state-space models make recurrence parameters input-dependent while using scan algorithms for parallel training; foundation models then require capability, efficiency, robustness, and safety evaluation.

$$h_t = \overline{A}(x_t)h_{t-1} + \overline{B}(x_t)x_t, \qquad y_t = C(x_t)h_t$$

Symbol	Meaning	Status
$\overline{A}(x_t)$	selective transition	input-dependent retention
$\overline{B}(x_t)$	selective write	input-dependent injection
$C(x_t)$	selective readout	state-to-output map

Selective state spaces, Mamba, and foundation-model evaluation: worked example

Quantity	Value	Meaning
sequence length	n	tokens
attention interactions	$O(n^2)$	dense pairwise
selective scan	$O(n)$	structured recurrence

What the result means

Adopt a scaled model only when capability gains survive length, system, robustness, and safety audits with a mitigation plan.

Selective state spaces, Mamba, and foundation-model evaluation: checking the result

Main idea

Selection lets content control what the state retains; linear sequence scaling changes the systems tradeoff but does not solve factuality, misuse, privacy, or distribution shift.

What to check

quality by length, throughput, calibration, and safety failure rate

Limit

efficient long-context processing can still retrieve the wrong evidence, memorize sensitive text, or generate unsafe actions

Selective state spaces, Mamba, and foundation-model evaluation: implementation

```
A = np.array([0.8, 0.6]); h = np.zeros(2); ys = []
for xt in np.linspace(-1, 1, 8):
    gate = 1/(1+np.exp(-3*xt))
    h = (A*(1-gate))*h + gate*np.array([xt, xt*xt])
    ys.append(h.sum())
print('Selective state outputs:', np.round(ys, 3))
```

Evidence and failure check

Check quality by length, throughput, calibration, and safety failure rate with the fitted model and data split; reject the run if efficient long-context processing can still retrieve the wrong evidence, memorize sensitive text, or generate unsafe actions.

Selective state spaces, Mamba, and foundation-model evaluation: limitations

Example

serving long-document assistance under memory and safety constraints

Explain the calculation and check quality by length, throughput, calibration, and safety failure rate, using a small example.

Common mistake

efficient long-context processing can still retrieve the wrong evidence, memorize sensitive text, or generate unsafe actions

What to do

Adopt a scaled model only when capability gains survive length, system, robustness, and safety audits with a mitigation plan.

Selective state spaces, Mamba, and foundation-model evaluation: check your understanding

Question

When can a linear-time selective state model replace quadratic attention, and what evidence is still required at scale?

Answer

Selection lets content control what the state retains; linear sequence scaling changes the systems tradeoff but does not solve factuality, misuse, privacy, or distribution shift.

Decision. Adopt a scaled model only when capability gains survive length, system, robustness, and safety audits with a mitigation plan.

Attention and state spaces spend computation differently

Decision

Dense attention exposes direct pairwise access; selective state spaces compress history through a recurrent state and scan.

Evidence

Compare quality and memory by sequence length on the target hardware.

Stop condition

Do not infer a winner from asymptotic complexity without measuring the dependency pattern of the task.

A causal mask needs a unit test

Decision

Change future tokens while keeping the prefix fixed and verify that all earlier logits remain unchanged.

Evidence

Test padding, causal, and combined masks separately before training.

Stop condition

Treat an unusually low training loss as a warning when the invariance test fails.

Generation methods make different promises

Decision

Compare VAE likelihood structure, GAN adversarial fidelity, and diffusion denoising under the same data and evaluation budget.

Evidence

Measure fidelity, coverage, conditioning, memorization, and latency separately.

Stop condition

Reject a model that wins one visual metric while collapsing diversity or copying training examples.

Long context is not the same as correct retrieval

Decision

Stratify tasks by evidence distance, number of distractors, and required reasoning depth.

Evidence

Report accuracy together with cited-evidence precision and latency.

Stop condition

Do not claim long-context competence from a single average score.

Caching changes the serving tradeoff

Decision

Measure prefill cost, per-token decode cost, memory growth, and batch efficiency for the intended context length.

Evidence

Profile the actual tokenizer and hardware configuration.

Stop condition

Shorten or restructure context when cache memory crowds out useful batch size.

Foundation-model evaluation needs a risk register

Decision

List factuality, privacy, bias, misuse, prompt injection, tool error, and over-reliance as separate failure modes.

Evidence

Assign an owner, test, threshold, and response to every high-impact risk.

Stop condition

Do not deploy on capability scores alone when a critical risk has no mitigation or monitoring path.

Choose an architecture using the task and measured performance

Decision

Start with the information path the task needs, then choose recurrence, attention, selective state space, or a hybrid.

Evidence

Compare quality, length scaling, robustness, safety, and cost on one decision sheet.

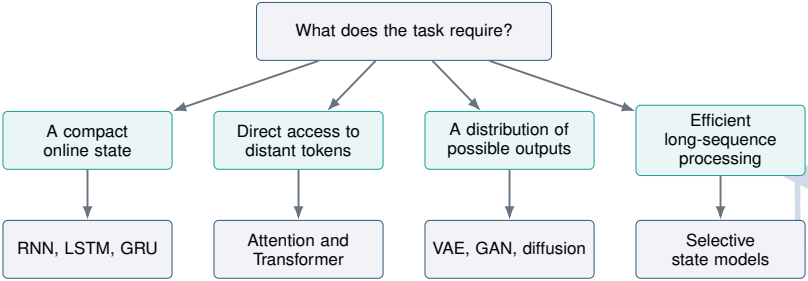
Stop condition

Keep an abstention or fallback route when the evidence is incomplete.

Compare model families by information flow and objective

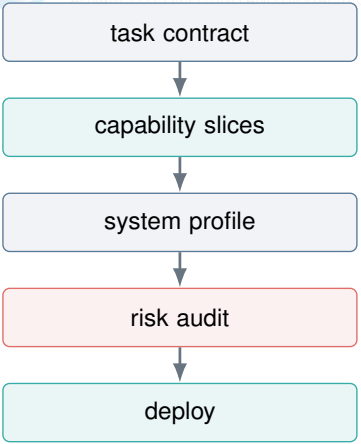
Family	Information path	Characteristic evidence
RNN / gated RNN	compressed recurrent state	length-stratified recall
Transformer	content-based pairwise interaction	mask tests and quadratic profile
VAE / GAN / diffusion	latent, adversarial, or denoising generation	fidelity, coverage, likelihood proxy
selective SSM	input-dependent linear recurrence	quality and throughput by length

Choose an architecture from the task requirements



These choices can be combined. Measure task quality, memory use, and latency on the same data and budget.

A reliable evaluation workflow



A benchmark score is one measurement, not a deployment license.

1. freeze causal and padding semantics
2. stratify by sequence length and difficulty
3. measure both fidelity and coverage for generation
4. profile memory and latency on target hardware
5. run robustness, privacy, and misuse tests
6. define monitoring, abstention, and rollback

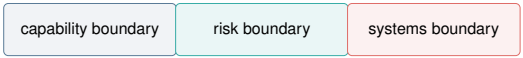
Measure capability subject to cost and risk

$$U = M_{task} - \lambda_c C_{compute} - \lambda_r R_{safety}, \qquad U_{deploy} > U_{baseline}$$

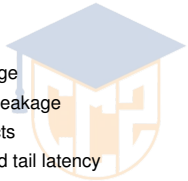
Observation	Likely issue	Next action
excellent short; poor long	information bottleneck	test gates, attention, or SSM
sharp but repetitive samples	coverage failure	measure generative recall
quality gain; unsafe tail	risk not controlled	mitigate or reject

Stress-test modern sequence systems

先进计算与数字工程研究中心
ADVANCED COMPUTING AND DIGITAL TECHNOLOGY RESEARCH CENTER



- perturb future tokens to test causality
- sweep context and dependency length
- separate generation fidelity and coverage
- search for memorization and sensitive leakage
- red-team misuse and instruction conflicts
- measure peak memory, throughput, and tail latency



Six takeaways to retain

先进计算与数字工程研究中心
ADVANCED COMPUTING AND DIGITAL TECHNOLOGY RESEARCH CENTER

1. recurrence compresses history into state
2. BPTT multiplies temporal Jacobians
3. gates create additive memory paths
4. attention retrieves content by query-key geometry
5. generative families optimize different evidence
6. efficient scaling does not replace safety evaluation

Exit ticket: defend a modern model choice

Prompt

Choose a recurrent, Transformer, generative, or selective-state approach. State the information path, training objective, one failure test, one system metric, and one safety gate.

Case

A long-document assistant needs 100k-token context, cited evidence, low latency, and a strict prohibition on revealing sensitive training examples.